LangChain v1.2.x: Deep Agents & Advanced Orchestration

Building Long-Running, Planning-Capable Agents

LangChain

March 2026

Agenda

Part I: Fundamentals

1. Mental Model: What are Deep Agents?
2. Problems They Solve
3. Architecture Overview
4. Core Tools
5. Building Your First Deep Agent

Part II: Advanced Patterns

1. Long-Running Tasks
2. Context Management
3. MCP Integration
4. Multi-Agent Orchestration
5. Monitoring & Fault Tolerance
6. Comparison Matrix
7. Pitfalls & Best Practices

45 slides total covering v1.2.x (March 2026)

Mental Model: Deep Agents

What are Deep Agents?

Autonomous agents capable of:

- **Long-running execution**: tasks that take minutes to hours
- **Multi-step planning**: decompose complex goals into subtasks
- **Persistent state**: offload context to filesystem
- **Sub-agent delegation**: spawn child agents for parallel work
- **Durable execution**: survive interruptions and failures

Think of them as mini-employees with planning skills, not just quick chatbot responses.

Standard Agent vs Deep Agent

Aspect	Standard Agent	Deep Agent
Execution Time	Seconds	Minutes to hours
Planning	Tool use loop	LLM + task planner
State Management	In-memory	Filesystem + DB
Task Decomposition	None	Automatic
Sub-agent Spawning	No	Yes
Fault Recovery	Loss of context	Persistent checkpoints
Context Size	Fixed prompt	Offloadable

Key Insight: Deep Agents trade speed for sophistication.

Problems Deep Agents Solve

Problem 1: Long-Running Tasks

Standard agents lose context during multi-minute operations. Deep Agents use persistent storage.

Problem 2: Context Explosion

Large research/data tasks overflow token limits. Solution: filesystem offloading.

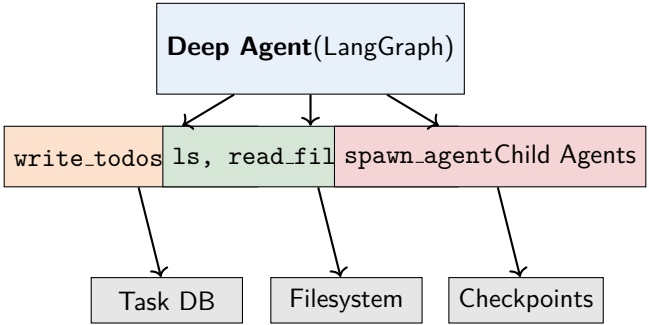
Problem 3: Parallel Work

Can't delegate subtasks to other agents. Solution: sub-agent spawning.

Problem 4: Fault Tolerance

Network glitch = start over. Deep Agents recover from checkpoints.

Deep Agent Architecture



[Docs: langchain-deepagents 0.1.x]

The write_todos Tool: Task Planning

```python
# Deep Agent creates a TODO list at start
from langchain_deepagents import write_todos

# Agent decides task decomposition
todos = write_todos([
  "Fetch latest market data from API",
  "Analyze price trends for 5 assets",
  "Generate trading signals",
  "Format report and save to disk",
])

# Agent tracks progress and can retry failed tasks
# Built-in retry logic for resilience
```

Key Features

- Automatic task breakdown from agent reasoning
- Progress tracking (pending, in-progress, completed)
- Sub-task dependencies and ordering
- Retry on failure with exponential backoff

Filesystem Tools for Context Management

```
# Read files (manage large context)
content = agent_tools.read_file("data/research.txt")

# List directory contents
files = agent_tools.ls("data/")

# Write or append to files
agent_tools.write_file(
  "data/analysis.txt",
  "Key finding: XYZ correlation..."
)

# Edit specific lines
agent_tools.edit_file(
  "data/report.md",
  start_line=10,
  end_line=15,
  new_content="Updated section..."
)
```

Sandbox Warning

Sub-Agent Spawning & Delegation

```python
# Parent agent can spawn child agents
from langchain_deepagents import spawn_child_agent

# Delegate a subtask
result = spawn_child_agent(
  goal="Analyze sentiment in customer reviews",
  context={
    "reviews_file": "data/reviews.txt",
    "output_file": "data/sentiment.json"
  },
  max_iterations=20,
  timeout_seconds=300
)

# Results passed back to parent
parent_agent.write_todos(
  f"Parent: sentiment analysis = {result.summary}"
)
```

Benefits

- **Parallelization**: multiple child agents work simultaneously

Building Your First Deep Agent

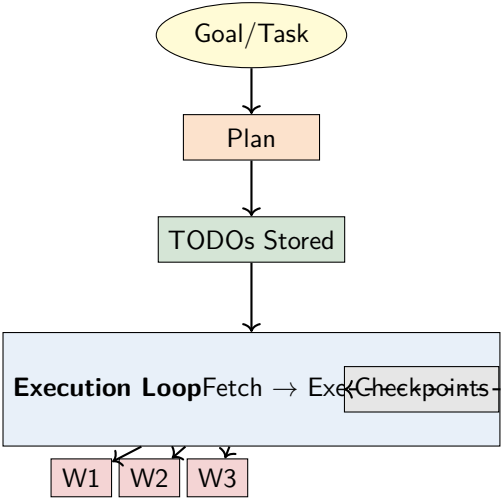
```python
from langchain import ChatOpenAI
from langchain_deepagents import DeepAgent
from langchain.tools import tool

# Define custom tools
@tool
def fetch_data(source: str) -> str:
  """Fetch data from source."""
  return f"Data from {source}"

# Create agent
llm = ChatOpenAI(model="gpt-4", temperature=0)
agent = DeepAgent(
  llm=llm,
  tools=[fetch_data],
  system_prompt="You are a data analysis agent...",
  max_iterations=30,
  checkpoint_dir="./checkpoints"
)

# Run for long task
result = agent.run(
  goal="Analyze Q1 sales data and forecast Q2"
)
```

Deep Agent Execution Flow

1. **Initialize**: Create agent with LLM + tools
2. **Plan**: Call LLM to decompose goal into subtasks
3. **Store Plan**: Save todos to database/filesystem
4. **Execute Loop**:
   - Fetch next pending task
   - Call tools to complete task
   - Save results and state
   - Mark task complete or retry
5. **Handle Failures**: If task fails, agent attempts recovery
6. **Report**: Final summary saved to file

All state is persistent. Network failure $\neq$ data loss.

Deep Agent Execution Architecture



Key: Multiple workers (sub-agents) execute in parallel. Checkpoints enable resumption.

Long-Running Task Patterns

Pattern 1: Batch Processing

```
# Process 1000 items in batches
for batch in batches_of(items, size=100):
  results = agent.process_batch(batch)
  agent.save_checkpoint()  # Persist every batch
```

Pattern 2: Periodic Reporting

```
# Long task with status updates every 5 min
while not agent.is_complete():
  agent.run_step()
  if time_elapsed % 300 == 0:
    agent.write_report("data/status.txt")
```

Timeout Handling

Always set `timeout_seconds` to prevent runaway agents.

Managing Large Context: Offloading to Filesystem

```python
# Instead of passing giant data in prompt
# Save to file and reference it
agent.write_file("data/dataset.json", large_data)

# In tools, reference the file
@tool
def analyze_data() -> str:
  """Analyze data from file."""
  data = agent.read_file("data/dataset.json")
  # Process data
  return "Analysis complete"

# Agent dynamically reads files as needed
# Keeps prompt tokens manageable
```

Prompt Size Limit

Don't exceed model's context window. Keep per-iteration messages under 10K tokens.

MCP (Model Context Protocol) Integration

What is MCP?

MCP is a standardized protocol for connecting external tools/servers to LLMs.

```python
# LangChain integrates MCP servers
from langchain_mcp import MCPClient

# Connect to external MCP server
client = MCPClient(
  server_url="http://localhost:3000",
  server_name="research-tools"
)

# MCP tools auto-available in agent
agent = DeepAgent(
  llm=llm,
  tools=[client.get_tools()],  # MCP tools
)
```

Examples

langchain-mcp-adapters: Auto Tool Conversion

```python
from langchain_mcp_adapters import (
  MCPToolAdapter, PromptMCPAdapter
)

# Automatically wrap MCP resources as tools
adapter = MCPToolAdapter(
  mcp_server_uri="mcp://localhost:3000"
)

# Discovery: find all available tools
tools = adapter.discover_tools()
print(f"Found {len(tools)} MCP tools")

# Create agent with auto-converted tools
agent = DeepAgent(
  llm=llm,
  tools=tools,  # Now LangChain-compatible
)
```

Benefit

Zero-code integration: any MCP server works automatically.

Multi-Agent Orchestration Patterns

Pattern 1: Supervisor Agent

One master agent delegates tasks to specialized workers.

Pattern 2: Swarm (Horizontal)

Multiple agents work in parallel, share results via message bus.

Pattern 3: Hierarchical

Tree structure: parent spawns children, children spawn grandchildren.

Pattern	Complexity	Best For
Supervisor	Low	Simple delegation
Swarm	Medium	Independent parallel tasks
Hierarchical	High	Complex decomposition

Supervisor Pattern Example

```
from langchain_deepagents import DeepAgent

# Supervisor delegates work
supervisor = DeepAgent(
  llm=llm,
  system_prompt="Delegate tasks to workers..."
)

# Specialized workers
analyst = DeepAgent(
  llm=llm,
  system_prompt="You analyze data..."
)
writer = DeepAgent(
  llm=llm,
  system_prompt="You write reports..."
)

# Supervisor spawns workers for subtasks
supervisor.tools.append(
  spawn_agent(analyst, "Analyze sales data")
)
supervisor.tools.append(
  spawn_agent(writer, "Write report")
```

Agent Handoff Patterns

```python
# Agent A detects it can't complete task
# Hands off to Agent B
from langchain.tools import tool

@tool
def handoff_to_specialist(task: str, agent_id: str):
  """Pass control to another agent."""
  # Find agent by ID
  recipient = get_agent(agent_id)
  # Pass context
  result = recipient.run(task)
  return result

# In execution:
# "This needs specialized knowledge" -> handoff
# Original agent pauses, specialist takes over
# Results flow back to original

# Key: all agents share same filesystem
# Easy state sharing between handoffs
```

Use Case

Durable Execution & Checkpointing

```
# Deep Agents auto-checkpoint after each task
agent = DeepAgent(
  llm=llm,
  tools=tools,
  checkpoint_dir="./checkpoints",
  checkpoint_interval=1  # Every task
)

# If interrupted, resume from checkpoint
# Load previous state
result = agent.resume_from_checkpoint(
  checkpoint_id="task-001"
)

# No work is lost!
# Agent knows which tasks are done
# Only retries pending/failed tasks
```

Design

Checkpoints saved to local filesystem (production: cloud storage recommended).

Monitoring Deep Agent Runs

```python
from langchain_deepagents import AgentMonitor

# Real-time monitoring
monitor = AgentMonitor()

# Track progress
monitor.on_task_start("Fetch data")
monitor.on_tool_call("fetch_api")
monitor.on_task_complete("Fetch data")

# Access metrics
stats = monitor.get_stats()
print(f"Tasks: {stats.completed}/{stats.total}")
print(f"Duration: {stats.elapsed_seconds}s")
print(f"Errors: {stats.error_count}")

# Integration with logging
agent.run(
  goal="...",
  monitors=[monitor]
)

# Export to JSON/CSV
monitor.export_metrics("metrics.json")
```

Comparison: Deep Agents vs Alternatives

Feature	Standard Agent	LangGraph	Deep Agent
Speed	***	***	*
Long-running	No	**	***
Planning	No	**	***
Checkpointing	No	***	***
Sub-agents	No	**	***
FS Context Mgmt	No	No	***
Learning curve	Easy	Medium	Medium

Choice Guide

- **Quick tasks (seconds)**: Standard Agent
- **Complex workflows**: LangGraph
- **Long-running + planning**: Deep Agent

When to Use Deep Agents

Good Fit

- Research tasks taking 10+ minutes
- Data processing with 100+ items
- Multi-stage workflows with handoffs
- Tasks requiring sub-task delegation
- Tasks needing fault tolerance

Poor Fit

- Real-time interactive chat
- Tasks under 30 seconds
- Simple tool calling
- Cost-sensitive applications

Pitfall 1: Runaway Agents

The Problem

Agent gets stuck in infinite loop or ignores goal completion.

```
# WRONG: No safeguards
agent = DeepAgent(llm=llm, tools=tools)
agent.run(goal="...")  # Could run forever!

# CORRECT: Set limits
agent = DeepAgent(
  llm=llm,
  tools=tools,
  max_iterations=50,       # Hard limit
  timeout_seconds=3600,    # 1 hour max
)
```

Best Practice

Always set both `max_iterations` and `timeout_seconds`.

Pitfall 2: Cost Explosion

The Problem

Long-running agents + token-priced LLMs = high costs.

```
# Monitor token usage
from langchain_core.callbacks import TokenUsageCallback

token_tracker = TokenUsageCallback()

agent.run(
  goal="...",
  callbacks=[token_tracker]
)

print(f"Tokens: {token_tracker.total_tokens}")
print(f"Cost: ${token_tracker.estimated_cost}")
```

Mitigation Strategies

- Use cheaper models (Haiku, Opus)
- Batch operations to reduce calls

Pitfall 3: Context Overflow

The Problem

Accumulating state → exceeds model's context window.

```
# WRONG: Concatenate everything into prompt
full_context = all_data + all_results + all_logs
agent.run(goal="...", context=full_context)

# CORRECT: Use filesystem
agent.write_file("data/all_results.json", results)
# In tools:
@tool
def summarize():
  """Summarize from file, not prompt."""
  data = agent.read_file("data/all_results.json")
  return summarize(data)
```

Rule of Thumb

Keep per-iteration context under 4K tokens. Offload the rest.

Pitfall 4: Unhelpful Error Messages

The Problem

Agent gets cryptic error, doesn't know how to recover.

```
# WRONG: Generic error
@tool
def fetch_data(url: str):
  response = requests.get(url)
  return response.json()

# CORRECT: Helpful recovery
@tool
def fetch_data(url: str):
  try:
    response = requests.get(url, timeout=10)
    return response.json()
  except requests.Timeout:
    return "ERROR: URL unreachable. Try again in 60s."
  except json.JSONDecodeError:
    return "ERROR: Invalid JSON. Check URL format."
```

Best Practice

Testing Deep Agents

```python
import pytest
from langchain_deepagents import DeepAgent

def test_agent_planning():
  """Test that agent creates reasonable plan."""
  agent = DeepAgent(llm=llm, tools=tools)
  plan = agent.create_plan("Analyze sales")

  assert len(plan.tasks) > 0
  assert all(isinstance(t, Task) for t in plan.tasks)

def test_agent_recovery():
  """Test checkpoint recovery."""
  agent1 = DeepAgent(llm=llm)
  agent1.run_step()
  ckpt_id = agent1.save_checkpoint()

  agent2 = DeepAgent(llm=llm)
  agent2.load_checkpoint(ckpt_id)
  assert agent2.current_task == agent1.current_task
```

Coverage Areas

Observability & Debugging

```
import logging

# Enable detailed logging
logging.basicConfig(level=logging.DEBUG)

# Structured logging from agent
agent = DeepAgent(
  llm=llm,
  tools=tools,
  verbose=True,  # Log every step
  log_file="agent.log"
)

# In logs you'll see:
# [PLAN] "Analyze data" -> [T1, T2, T3]
# [EXECUTE] T1 "Fetch from API"
# [CALL] tool=fetch_api args={...}
# [RESULT] "200 records retrieved"
# [SAVE] checkpoint-001

# Parse logs for debugging
with open("agent.log") as f:
  for line in f:
    if "[ERROR]" in line:
```

Cost Optimization Strategies

1. **Model Selection**: Use Haiku for planning, Opus for reasoning
2. **Batching**: Process multiple items per iteration
3. **Caching**: Store tool results, reuse within session
4. **Timeouts**: Prevent runaway agents
5. **Sampling**: Test on 10

```
# Example: Use cheaper model for planning
planner_llm = ChatOpenAI(model="gpt-3.5-turbo")
executor_llm = ChatOpenAI(model="gpt-4")

agent = DeepAgent(
  planner_llm=planner_llm,
  executor_llm=executor_llm,
  tools=tools
)
```

Typical 1-hour agent run: $0.50-$5.00 depending on model choice.

Production Deployment Checklist

Before Deployment

- Set `max_iterations` and `timeout_seconds`
- Enable checkpointing to cloud storage (S3, GCS)
- Configure logging to centralized system
- Set up alerts for errors/cost spikes
- Test recovery from failures
- Document all custom tools and expected behaviors
- Implement rate limiting on tool calls
- Add monitoring dashboards

Recommended Stack

Agent + Cloud Storage + Logging Service + Monitoring = production-ready.

Real-World Example: Research Agent

Goal

Autonomous agent: investigate a company, write analysis report.

```
from langchain_deepagents import DeepAgent
from langchain.tools import tool

@tool
def web_search(query: str) -> str:
  """Search web for company info."""
  # Use search API
  return results

@tool
def fetch_financials(ticker: str) -> str:
  """Fetch latest financials."""
  # Use financial API
  return data

# Agent
agent = DeepAgent(
  llm=ChatOpenAI(model="gpt-4"),
  tools=[web_search, fetch_financials]
```

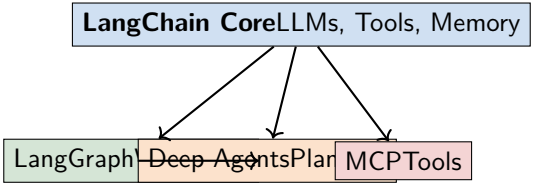
Real-World Example: Data Processing Pipeline

Goal

Process 10,000 customer records: clean, enrich, analyze.

```python
# Deep Agent with sub-task delegation
supervisor = DeepAgent(
  llm=llm,
  system_prompt="Manage data processing pipeline"
)

# Create batch-processing tool
@tool
def spawn_batch_worker(batch_id: int, records):
  """Delegate batch to worker agent."""
  worker = DeepAgent(
    llm=llm,
    system_prompt="Process records batch"
  )
  return worker.run(
    goal=f"Clean and enrich batch {batch_id}",
    context={"records": records}
  )
```

How Deep Agents Fit in LangChain Ecosystem



Deep Agents built on LangGraph for durable execution.
Integrate MCP servers as tools via adapters.
Use standard LangChain tools throughout.

Summary: Deep Agents at a Glance

What They Are

Autonomous, long-running agents with planning, sub-task delegation, and durable execution.

When to Use

Multi-step research, data processing, complex workflows requiring hours of work.

Core Tools

- `write_todos`: Task planning
- Filesystem tools: Context management
- `spawn_agent`: Sub-task delegation
- Checkpointing: Fault tolerance

Key Benefits

Cheat Sheet: Deep Agent Best Practices

Practice	Guideline
Always set limits	`max_iterations` + `timeout_seconds`
Offload large context	Use filesystem, not prompt
Monitor costs	Track token usage per run
Checkpoint strategy	Enable for any task $>$ 5 minutes
Error handling	Provide actionable error messages
Testing	Test planning, execution, recovery
Logging	Enable verbose logging in production
Timeouts	Per-tool + per-agent timeouts

Remember: Deep Agents trade speed for sophistication. Use wisely!

Self-Check Questions

1. What is the primary difference between a Standard Agent and a Deep Agent?
   - Answer: Deep Agents support long-running, planned, resilient execution with sub-task delegation.
2. How do Deep Agents prevent context overflow?
   - Answer: Filesystem offloading; save large data to files, reference in tools.
3. What tool is used for initial task decomposition?
   - Answer: `write_todos` creates an ordered task list.
4. How do you recover from agent interruption?
   - Answer: Load from checkpoint; agent resumes from last completed task.

Self-Check Questions (Continued)

5. Name two sub-agent spawning use cases.
   - Answer: Batch processing (10K items), parallel analysis (5 domains).
6. What is the Supervisor pattern?
   - Answer: One master agent delegates work to specialized worker agents.
7. How does MCP integration work in LangChain?
   - Answer: Use langchain-mcp-adapters; MCP servers auto-convert to LangChain tools.
8. What are the top 3 pitfalls to avoid?
   - Answer: Runaway agents (no limits), cost explosion (track tokens), context overflow (offload).

Resources & References

Official Documentation

- LangChain: `https://python.langchain.com`
- langchain-deepagents: `https://github.com/langchain-ai/langchain-deepagents`
- LangGraph: `https://langchain-ai.github.io/langgraph`
- MCP Spec: `https://modelcontextprotocol.io`

Recommended Reading

- "Planning and Replanning" (Anthropic, 2024)
- "Fault-Tolerant AI" whitepaper
- LangGraph Docs: Checkpoint Strategies

Quick Start: Template

```python
from langchain import ChatOpenAI
from langchain_deepagents import DeepAgent
from langchain.tools import tool

@tool
def my_tool(arg: str) -> str:
  """Do something useful."""
  return f"Result: {arg}"

llm = ChatOpenAI(model="gpt-4")
agent = DeepAgent(
  llm=llm,
  tools=[my_tool],
  system_prompt="You help with...",
  max_iterations=30,
  timeout_seconds=3600,
  checkpoint_dir="./checkpoints"
)

result = agent.run(goal="Your task here")
print(result.summary)
```

Next Step: Customize tools, system prompt, and goal for your use case.

Common Extensions & Customizations

Extension 1: Custom Tool Wrappers

Wrap external APIs with Deep Agent semantics (retries, logging).

Extension 2: Multi-Model Delegation

Route tasks to different models (Haiku for fast, Opus for hard).

Extension 3: External Memory

Integrate vector DB for semantic search across past results.

Extension 4: Feedback Loops

Human-in-the-loop: agent asks for approval before risky actions.

LangChain Philosophy: Composable building blocks. Mix and match!

Troubleshooting Common Issues

Issue: Agent loops forever

Solution: Set `max_iterations`. Check system prompt clarity.

Issue: High cost per run

Solution: Use cheaper model. Enable caching. Batch operations.

Issue: Agent forgets context

Solution: Save to filesystem. Use shorter iteration windows.

Issue: Tool fails

Solution: Add detailed error messages. Implement retry logic.

Issue: Checkpoint loading fails

Deep Agents Roadmap (2026+)

Q2 2026 Planned

- GPU acceleration for planning
- Multi-tenancy support
- Web UI for agent monitoring
- Advanced scheduling (cron-like)

Beyond

- Federated agent networks
- Formal verification of plans
- Autonomous cost optimization
- Self-improving agents

Current Status: v0.1.x stable, production-ready. Active development.

Community & Support

Getting Help

- **GitHub Issues**: Report bugs, request features
- **Discord**: Real-time community chat (`discord.gg/langchain`)
- **GitHub Discussions**: Best practices, architecture Q&A
- **Stack Overflow**: Tag with `langchain`

Contributing

- PRs welcome for bugs and features
- Docs improvements highly valued
- Tool implementations appreciated

Share your projects! Community builds amazing things.

Closing Remarks

The Big Picture

Deep Agents represent a shift toward **truly autonomous AI systems**:

- Not just answering questions
- But **planning**, **executing**, and **learning**
- Over **hours**, not milliseconds
- With **resilience** and **transparency**

The future of AI agents is long-running, resilient, and plan-driven.

Now go build something amazing with Deep Agents!